Session Types, Simply

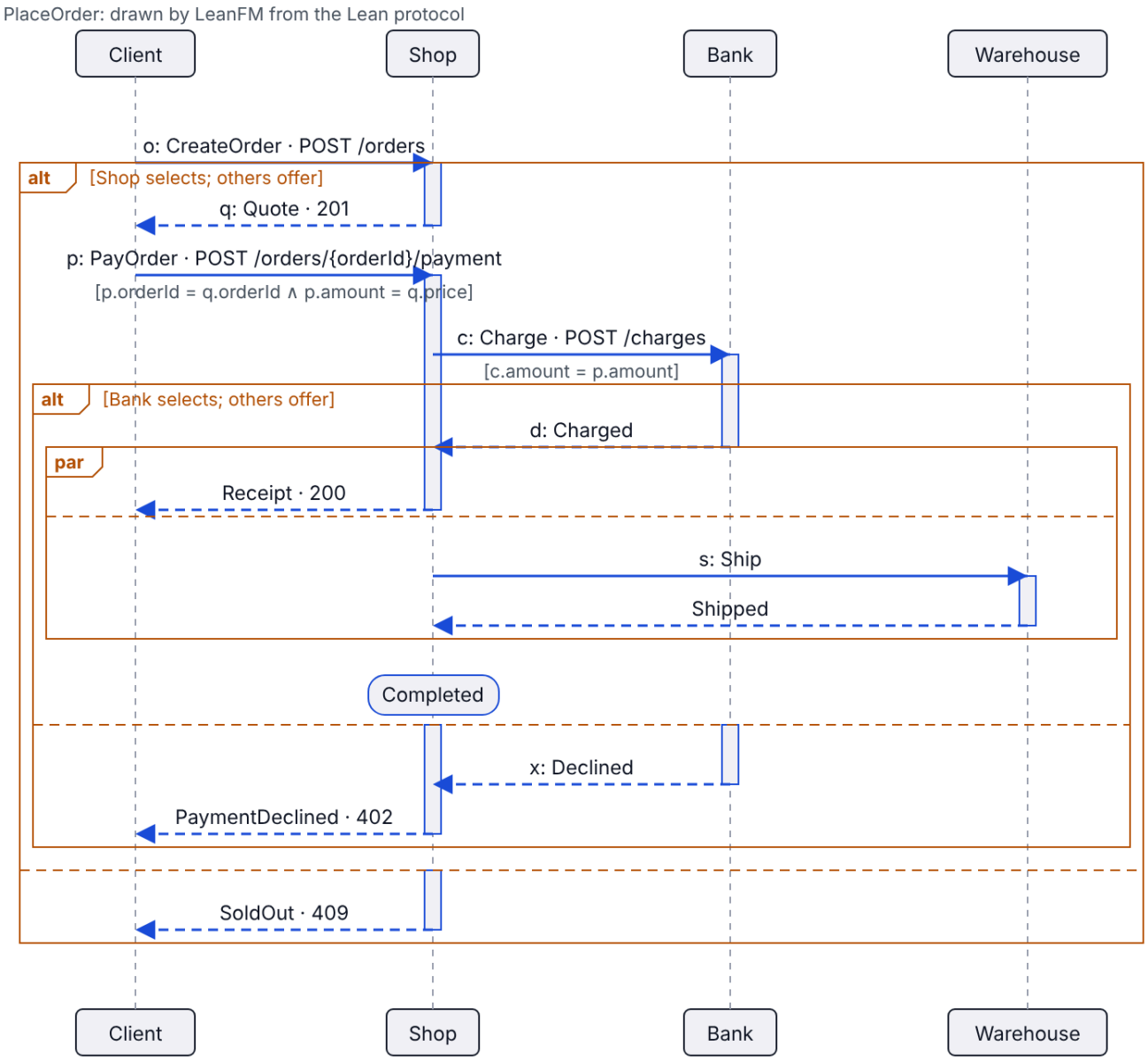
Nov 7, 2026 · @Robert Jason Fielding

Every team already draws interaction diagrams and keeps an OpenAPI file. A session type makes the diagrams checkable. It keeps every path, method and status code in the OpenAPI file, and adds what OpenAPI cannot say: the order of calls, the data that must match between them, who decides each outcome, and what each handler calls downstream. This page refines a small OpenAPI description in two layers: protobuf gives each message its structure, and a session type in checked Lean gives the conversation its structure. Then it explains the six ideas behind it.

Interaction diagrams are everywhere

Every design review has one: actors across the top, arrows down the page. It is the most common picture in software engineering, and a session type is a set of them, made precise. Engineers already use them in five ways, and each becomes something LeanFM checks:

What engineers already draw	What it becomes
Scenarios grouped by feature: place an order, check an order	protocols grouped into one service with `sessions` and `choice`
A box that says “see the payment diagram”	`include`: a nested protocol, refined on its own
A request, then a dashed reply marked 200 or 402	a message and its response, tied by `cause`; the status code is the branch
alt and par frames	`choice`, decided by one actor, and `par` or `join`
What each lifeline does over time	that actor's state machine, derived by projection



PlaceOrder, drawn by LeanFM from the protocol later in this paper, not by hand. Each alt frame names the actor that selects; the others offer. Each call is an activation bar on the callee, from its request to its response, and its route and status codes are on the arrows. Dashed arrows are responses: their cause chain reaches a request from the receiver to the sender, so Receipt and PaymentDeclined answer PayOrder even though the bank spoke in between, while PayOrder, a new request made after the Quote, is solid.

The drawing is generated because nothing in the protocol is missing from a sequence diagram, and nothing in a sequence diagram is missing from the protocol. Each construct has one notation:

In the Lean protocol	In the diagram
`x: L from A to B`	an arrow from A to B labelled `x: L`
`route "POST /orders"`	`· POST /orders` after the label
`cause y`	a dashed response when the chain of causes reaches a request from B to A; otherwise a solid arrow
`status 402`	`· 402` after the response's label
a request and its responses	an activation bar on the callee, from the request to the response
`where e`	`[e]` under the arrow
`L at A`	an action box on A's lifeline
`choice { } or { }`	an `alt` frame. The actor that starts every branch **selects**; every other actor **offers**, waiting to learn the branch from the first message it receives
`par { } and { }`	a `par` frame, branches split by dashed lines
`join k of { } and { }`	a `par` frame captioned *any k of n*
`rec N { … continue N; }`	a `loop N` frame
`sessions { }`	a `loop` frame around one instance: one per request, concurrently
`include P …`	a `ref P` frame around P's steps
a lifeline	that actor's local type, from projection: its sends, receives, selects and offers

Select and offer are the two halves of an alt frame. In the textbook they are separate operators, ⊕ and &, attached to local types; in a diagram they are the same frame seen from two lifelines. The only thing the frame has to record is who decides, and LeanFM finds that rather than asking for it. A choice no single actor starts is drawn anyway, captioned as a race, because that is the bug.

What the diagrams leave out is the queue. Every arrow ends in the recipient's inbox, which is bounded, and some senders also buffer what they send: a TCP send buffer, a Kafka producer batch, an outbox table. Sending, delivery and consumption are then three separate steps. LeanFM keeps all three: a full outbound queue blocks the sender, a full inbox blocks delivery, and each actor takes one message at a time. Those bounds keep every check finite, and they expose bugs a diagram hides, such as two services each blocked on the other's full queue.

Start from OpenAPI

A small shop has three operations: create an order, pay for it, look it up. Its OpenAPI description lists each path, method, request body and possible response:

```
openapi: 3.1.0
info: { title: Shop, version: 1.0.0 }
paths:
  /orders:
    post:
      operationId: createOrder
      requestBody: { $ref: '#/components/requestBodies/CreateOrder' }
      responses:
        '201': { $ref: '#/components/responses/Quote' }
        '409': { $ref: '#/components/responses/SoldOut' }
  /orders/{orderId}/payment:
    post:
      operationId: payOrder
      requestBody: { $ref: '#/components/requestBodies/PayOrder' }
      responses:
        '200': { $ref: '#/components/responses/Receipt' }
        '402': { $ref: '#/components/responses/PaymentDeclined' }
  /orders/{orderId}:
    get:
      operationId: getOrder
      responses:
        '200': { $ref: '#/components/responses/OrderStatus' }
        '404': { $ref: '#/components/responses/NotFound' }
components:
  schemas:
    CreateOrder:
      properties: { item: { type: string } }
    Quote:
```

```
      properties: { orderId: { type: integer }, price: { type: integer } }
    SoldOut:
      properties: { item: { type: string } }
    PayOrder:
      properties: { orderId: { type: integer }, amount: { type: integer } }
    Receipt:
      properties: { orderId: { type: integer } }
    PaymentDeclined:
      properties: { orderId: { type: integer } }
    OrderStatus:
      properties: { orderId: { type: integer }, paid: { type: boolean } }
    NotFound:
      properties: { orderId: { type: integer } }
  # requestBodies and responses wrap each schema as application/json
```

This is a good contract for one call at a time. A client generator can build every request, and a server stub can return every status code.

What the OpenAPI file cannot say

OpenAPI describes each operation on its own. Everything that connects the operations is left to prose, or to nobody:

Missing from OpenAPI	Example in the shop
Order of calls	a payment comes after a quote, and never after SoldOut
Data that must match	the payment's orderId and amount must equal the quote's
Who decides each outcome	the shop decides 201 or 409; the bank's answer decides 200 or 402
What a handler calls downstream	the payment handler charges the bank and waits for it
Why an error happened	a 402 is caused by the bank's Declined, which was caused by this payment

These are exactly the facts teams argue about in incident reviews. A session type writes them down, and a checker tests them.

Protobuf gives each message its structure

A protocol is built from messages, and each message needs a precise shape: its fields, their types, and how it is encoded on the wire. Protobuf is a good abstraction for that layer. It is language-neutral, compact on the wire, and safe to evolve, because fields are identified by number rather than by position or name.

The shop's messages, as a `.proto` file that `protoc` accepts:

```
syntax = "proto3";
package shop.v1;

// One message per word of the protocol.
message CreateOrder     { string item = 1; }
message Quote           { uint32 order_id = 1; uint32 price = 2; }
message SoldOut         { string item = 1; }
message PayOrder        { uint32 order_id = 1; uint32 amount = 2; }
message Charge          { uint32 amount = 1; }
message Charged         {}
message Declined        { string reason = 1; }
message Receipt         { uint32 order_id = 1; }
message PaymentDeclined { uint32 order_id = 1; }
message GetOrder        { uint32 order_id = 1; }
message OrderStatus     { uint32 order_id = 1; bool paid = 2; }
message NotFound        { uint32 order_id = 1; }
message Ship            { uint32 order_id = 1; }
message Shipped         { uint32 order_id = 1; }
message Completed       { uint32 order_id = 1; }

// Every message in the service is one case of this envelope.
message ShopMessage {
  oneof atom {
    CreateOrder     create_order     = 1;
    Quote           quote            = 2;
    SoldOut         sold_out         = 3;
    PayOrder        pay_order        = 4;
    Charge          charge           = 5;
    Charged         charged          = 6;
    Declined        declined         = 7;
    Receipt         receipt          = 8;
    PaymentDeclined payment_declined = 9;
    GetOrder        get_order        = 10;
    OrderStatus     order_status     = 11;
```

```
    NotFound        not_found        = 12;
    Ship            ship             = 13;
    Shipped         shipped          = 14;
    Completed       completed        = 15;
  }
}

// One recorded event. prior links back to the events it comes right
// after; cause_id names the message it answers.
message Event {
  string          id       = 1;
  repeated string prior    = 2;
  string          cause_id = 3;
  string          session  = 4;
  string          src      = 5;
  string          dst      = 6;
  ShopMessage     message  = 7;
}
```

Four details matter for what follows:

- **The envelope's `oneof` is the vocabulary.** Every message the service can send is one case of `ShopMessage`, so the grammar's words are exactly its cases. A receiver can always tell which message it got, which is what rule 2 below depends on.
- **`prior` is a backwards linked list.** Each event lists the events it comes right after. One entry means plain sequence. Two events with the same prior happened concurrently. An event with several priors is where concurrent branches join. Following `prior` back from any event gives its causal past.
- **`cause_id` makes a response belong to its request.** It is `cause` from the protocol, carried on the wire. It is not the same link as `prior`: a response may come right after other work, but it still answers one particular message.
- **The JSON mapping matches the OpenAPI file.** Protobuf's standard JSON mapping turns `order_id` into `orderId`, which is the field name the OpenAPI schemas use. The numbers are 32-bit because the mapping writes 64-bit integers as JSON strings, which would not match the OpenAPI integer type.

The result is three layers, each answering one question:

Layer	Answers	In the shop
Protobuf	what each message contains, and how it is encoded	`PayOrder` has an order ID and an amount
Session type	which message may follow which, who decides, and which response answers which request	PayOrder comes only after a Quote, and is answered by Receipt or PaymentDeclined
OpenAPI	how the conversation travels over HTTP	POST /orders/{orderId}/payment returns 200 or 402

The OpenAPI file is the thinnest layer. Given the messages and the conversation, plus a choice of path and method for each request, it can be derived instead of written by hand.

The same service as session types, in Lean

Each protobuf message has a Lean structure with the same fields, so the protocol's conditions are type-checked against real field names. A misspelt message name is a compile error, not a new message:

```
structure CreateOrder where item : String deriving TypeName
structure Quote where
  orderId : Nat
  price : Nat
  deriving TypeName
structure PayOrder where
  orderId : Nat
  amount : Nat
  deriving TypeName
structure Charge   where amount : Nat    deriving TypeName
structure Charged  where                 deriving TypeName
structure Declined where reason : String deriving TypeName
-- SoldOut, Receipt, Ship, Shipped, Completed, ... likewise
```

Each operation becomes a protocol: a request that names its `route`, then a choice of responses. Each response names its request with `cause` and carries its `status`, so the status code is the branch:

```
protocol PlaceOrder (Client, Shop, Bank, Warehouse) {
  o: CreateOrder from Client to Shop route "POST /orders";
  choice {
    q: Quote from Shop to Client cause o status 201;
    p: PayOrder from Client to Shop cause q
         route "POST /orders/{orderId}/payment"
         where p.orderId = q.orderId ∧ p.amount = q.price;
    -- the payment handler calls the bank
    c: Charge from Shop to Bank cause p route "POST /charges"
         where c.amount = p.amount;
    choice {
      d: Charged from Bank to Shop cause c;
      par {
         Receipt from Shop to Client cause d status 200;
      } and {
         s: Ship from Shop to Warehouse cause d;
            Shipped from Warehouse to Shop cause s;
      }
      Completed at Shop;
    } or {
      x: Declined from Bank to Shop cause c;
         PaymentDeclined from Shop to Client cause x status 402;
    }
  } or {
       SoldOut from Shop to Client cause o status 409;
  }
}

protocol CheckOrder (Client, Shop) {
  g: GetOrder from Client to Shop route "GET /orders/{orderId}";
  choice {
    OrderStatus from Shop to Client cause g status 200;
  } or {
    NotFound from Shop to Client cause g status 404;
  }
}
```

Session types also say what may happen at the same time. Once the bank has charged the card, `par` lets the shop answer the client and ship the order concurrently: neither side waits for the other, and each keeps its own order. `Completed at Shop` comes after both sides finish, so it is a join. The warehouse is an internal service, so it never appears in the

public OpenAPI file, yet the protocol still says when it is called. A threshold join is written `join 2 of { … } and { … } and { … }`: it continues after any two branches, as a write to three replicas does once two have stored it.

The service groups the operations, as the OpenAPI file does. `sessions` means any number of independent calls at once, as a server handles them; `choice` means each call is one of the operations; `include` nests a protocol inside another:

```
protocol ShopService (Client, Shop, Bank, Warehouse) {
  sessions {
    choice {
      include PlaceOrder Client Shop Bank Warehouse;
    } or {
      include CheckOrder Client Shop;
    }
  }
}
```

Every row of the previous table is now one line of code. The routes and status codes are on the lines they label. The order of calls is the order of lines. The matching data is the `where` condition, which Lean type-checks. The deciders follow from who sends first in each branch. The downstream call is the `Charge` line. The reason for a 402 is its `cause` chain: PaymentDeclined, Declined, Charge, PayOrder.

What the checker says

LeanFM reports no issues for all three protocols. They pass the two buildability rules explained below, and the HTTP rule: every response goes back to the caller that is waiting for it. It also derives each party's part. The client's part of `PlaceOrder` is exactly what a client SDK must do:

```
send CreateOrder to Shop;
offer {
  receive Quote from Shop;
  send PayOrder to Shop;
  offer {
    receive Receipt from Shop;
  } or {
    receive PaymentDeclined from Shop;
  }
} or {
  receive SoldOut from Shop;
}
```

Two plausible mistakes, and what LeanFM reports for each:

Mistake	LeanFM reports
The 402 is forgotten: on Declined the shop never answers the client	`Client cannot tell which branch Bank chose`
The bank sends the Receipt straight to the client	`Receipt is a request from Bank to Client, which Bank may not call`

The first leaves a client waiting forever for a response that never comes. The second cannot happen over HTTP at all, because the client never called the bank. An OpenAPI file can describe both services without noticing either problem.

Calls and typestate, from cause

`cause` is what turns messages into calls. A request, the detail it sets off, and the response that names it make one synchronous call: an HTTP operation with its status codes, or a function from its parameters to its return value. LeanFM reads the calls straight off the protocol. A message whose chain of causes reaches a request from its receiver to its sender is a response; everything else is a request. The detail of a call is everything its request causes, except its responses and what they cause, so a call made inside another one is listed under it:

```
o: CreateOrder  Client → Shop  POST /orders
  returns Quote 201 | SoldOut 409
p: PayOrder  Client → Shop  POST /orders/{orderId}/payment
  returns Receipt 200 | PaymentDeclined 402
    c: Charge  Shop → Bank  POST /charges
      returns Charged | Declined
    s: Ship  Shop → Warehouse
      returns Shipped
g: GetOrder  Client → Shop  GET /orders/{orderId}
  returns OrderStatus 200 | NotFound 404
```

That is the OpenAPI file again, written by LeanFM: the public operations at the top level, and the calls the shop makes while serving a payment indented under it, including the warehouse call the public file never mentions.

Each party's local type is also a *typestate* API. Every state the party can be in is a type of its own, and each method uses up the state it is called on and returns the next one, as in `Began.doAuth(b : User) : LoggedIn`. The client's part of `PlaceOrder`:

```
start : S0
S0.sendCreateOrder(m : CreateOrder) : S1    -- to Shop
S1.receive() : Quote × S2 (from Shop) | SoldOut × End (from Shop)
S2.sendPayOrder(m : PayOrder) : S3    -- to Shop
S3.receive() : Receipt × End (from Shop) | PaymentDeclined × End (from Shop)
```

A client holding `S1` can only wait for the shop's answer, and a client holding `End` has nothing left to call. Paying twice would mean calling `sendPayOrder` on an `S2` already used up. In a language with linear types the compiler rejects that; in others a generated SDK hides the old state so the call cannot be written.

`cause` gives the conversation its structure; the `prior` links of a recorded run give its timing. `prior` is a backwards linked list of events, so it records what actually happened before what, including which events were concurrent. A protocol's `par` is the promise that some events may be concurrent; the `prior` links of a run show whether they were.

One recorded run: prior and cause

Here is one successful run of `PlaceOrder` as `Event` records. Each event points back to the events it comes right after (`prior`), and to the message it answers (`cause_id`):

id	Message	From → to	prior	cause_id
e1	CreateOrder	Client → Shop	(none)	(none)
e2	Quote	Shop → Client	e1	e1
e3	PayOrder	Client → Shop	e2	e2
e4	Charge	Shop → Bank	e3	e3
e5	Charged	Bank → Shop	e4	e4
e6	Ship	Shop → Warehouse	e5	e5
e7	Receipt	Shop → Client	e5	e5
e8	Shipped	Warehouse → Shop	e6	e6
e9	Completed	at Shop	e7, e8	(none)

The `prior` links carry the concurrency. e6 and e7 share the prior e5, so they are the two sides of the `par`, and neither depends on the other. e9 has two priors, so it is the join. A log may list e6, e7 and e8 in any of three orders; all three are the same run, because the links, not the line order, say what depends on what.

The `cause_id` links carry the request and response pairing. Following them back from e7 reaches e3, the PayOrder request: the Receipt is the 200 for that call.

The rest of this page explains why this works, in six ideas. You will not need the textbook theory to follow it.

Why session types

The worst bugs between services sit at the boundary, not inside any one service. A message arrives that the receiver has no handler for. A service waits forever for a reply nobody owes it. Two services each think the other one decides.

Without a contract, the only way to know what the other side will send is to read its code. A session type is that contract: the whole conversation, written once, from nobody's point of view. Each party then checks its own part against it, without reading anyone else's code.

The idea is about 30 years old and still rare in industry. The usual explanations need a process calculus, typing judgments and a projection operator before the first useful example. None of that machinery is needed to use the idea.

The textbook version, in one paragraph

You will meet this elsewhere, so here it is once. In multiparty session types (Honda, Yoshida and Carbone, 2008), a *global type* such as p → q : ℓ(S).G describes the whole conversation. *Projection*, a partial function with a *merge* operator for branches, derives one *local type* per role, built from sends (q!ℓ) and receives (p?ℓ). Programs are written in a typed π-calculus, and a typing judgment checks each process against its local type. *Subject reduction* then yields three theorems: communication safety, session fidelity and progress. Channels are asynchronous, unbounded and lossless. Deciding when one local type may replace another (asynchronous subtyping) is undecidable in general.

Nothing below depends on this paragraph. The table near the end maps each of its terms to the plain version.

1. A protocol is a grammar whose words are messages

A grammar lists the legal sentences of a language. A protocol lists the legal conversations of a system. The only difference is the words: in a protocol, each word is one message, with a sender and a receiver.

`PlaceOrder` above, written in LeanFM's notation, is such a grammar. It has three kinds of legal conversations:

1. CreateOrder, Quote, PayOrder, Charge, Charged, then Receipt alongside Ship and Shipped, then Completed. The concurrent part can interleave three ways.
2. CreateOrder, Quote, PayOrder, Charge, Declined, PaymentDeclined.
3. CreateOrder, SoldOut.

The constructs are the ones every grammar has:

- **Sequence:** one line after another.
- **Choice:** `choice { … } or { … }`; exactly one branch happens.
- **Repetition:** a named block that `continue` repeats.
- **Parallel:** `par { … } and { … }`; both happen, interleaved.

Two additions carry data. `cause o` says this message answers `o`; over HTTP, that is what makes a response belong to its request. `where p.amount = q.price` is a condition on payloads; it is how the grammar ties a later message to an earlier one.

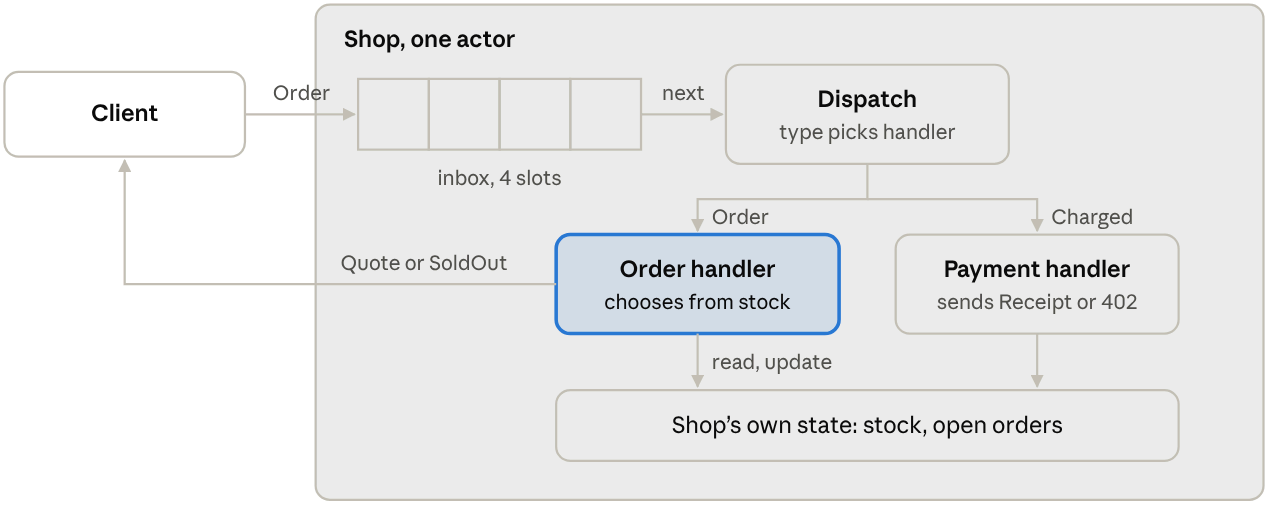
That is the whole of what the textbook calls a global type. A recorded run of the real system is correct when it is a sentence of this grammar, which can be checked from logs alone.

2. Every party is a state machine that reacts

Each party in the conversation is an *actor*: a state machine with an inbox of fixed size, and sometimes an outbox of fixed size too. It repeats one loop:

1. Take the next message from the inbox.
2. The message picks the handler. A `CreateOrder` runs the order handler; a `Charged` runs the payment handler.
3. The handler updates the actor's state, and may put messages in other actors' inboxes.

A message picks the handler; the choice uses only the shop's own state



one actor's loop · inbox, dispatch, handlers, own state

The shop never sees the client's state, only the messages in its inbox; its reply depends on its own stock.

That is how real servers, mobile apps and embedded controllers already work. It also settles who makes choices. **Every choice is made inside one actor, from what that actor has seen.** In PlaceOrder, only the shop knows its stock, so the shop decides between

Quote and SoldOut. The client cannot decide for it; the client learns the decision when Quote or SoldOut arrives.

Nothing is chosen globally. There is no shared coin that everyone sees at the same instant. An actor can only affect another by putting a message in its inbox. This is the physical rule the rest of the method depends on.

3. What each party owes is the grammar, filtered

To find what one actor must do, delete every message it neither sends nor receives. What remains is its part, written as sends and receives. The shop's part of PlaceOrder:

```
receive CreateOrder from Client;
select {
  send Quote to Client;
  receive PayOrder from Client;
  send Charge to Bank;
  offer {
    receive Charged from Bank;
    par {
      send Receipt to Client;
    } and {
      send Ship to Warehouse;
      receive Shipped from Warehouse;
    }
    Completed;
  } or {
    receive Declined from Bank;
    send PaymentDeclined to Client;
  }
} or {
  send SoldOut to Client;
}
```

The bank's part:

```
offer {
  receive Charge from Shop;
  select {
    send Charged to Shop;
  } or {
    send Declined to Shop;
  }
} or {
}
```

The two choice words say who decides. `select` is a choice this actor makes: the shop selects 201 or 409, and the bank selects Charged or Declined. `offer` is someone else's choice, which this actor learns from what arrives. The bank's second branch is empty: when the shop sells out, the bank is never involved.

The textbook calls this *projection*, and each filtered part a *local type*. In practice it is the handler code a team owes for one conversation, and the test it must pass.

The two rules that make a protocol buildable

Not every grammar can be built from actors that only see their own inboxes. Two checks rule out the ones that cannot. Both follow from idea 2.

Rule 1: one actor starts every branch of a choice. That actor is the one deciding. A first draft of a file transfer lets the sender's timer start one branch and the receiver's acknowledgement start the other. Both can happen at once, so the protocol is a race. LeanFM reports:

```
no single role decides a choice, so it is a race: Sender starts Timeout;
Receiver starts Ack
```

Rule 2: anyone whose future depends on the choice must be told by a message. Here is the forgotten 402 from the checker's table. When the bank declines, the shop never answers the client:

```
protocol PlaceOrderNo402 (Client, Shop, Bank) {
  o: CreateOrder from Client to Shop;
  choice {
    q: Quote from Shop to Client cause o;
    p: PayOrder from Client to Shop cause q;
    c: Charge from Shop to Bank cause p;
    choice {
      d: Charged from Bank to Shop cause c;
         Receipt from Shop to Client cause d;
    } or {
         Declined from Bank to Shop cause c;
    }
  } or {
       SoldOut from Shop to Client cause o;
  }
}
```

The client has paid and waits for Receipt, which never comes. LeanFM reports:

```
Client cannot tell which branch Bank chose
```

The correct `PlaceOrder` passes rule 2 even though the bank is never told about SoldOut. The bank's part starts when Charge arrives; in the SoldOut branch it never starts, so it owes nothing. The textbook projection rejects that case. This version accepts it because that is how servers work: a handler that was never invoked has nothing to clean up.

The textbook names for these are *realizability* or *non-local choice* (rule 1) and *projectability* (rule 2).

4. Channels have physics

The textbook assumes every message arrives exactly once, in order, through a queue that never fills. Real channels do none of that, so each channel's behaviour is declared alongside the protocol:

Property	What gets declared
Capacity	the inbox size, an optional outbox size, and what happens when either is full: block the sender, reject, drop the newest, or drop the oldest

Property	What gets declared
Delivery	ordered or not; lost, duplicated, or neither
Time	timers, which may fire before the reply they wait for has had time to arrive
Failure	lost messages, restarted actors, expired sessions

The physics changes what the same grammar means. The original TFTP sender (RFC 783) and the corrected one (RFC 1123) follow the same lock-step grammar, and both pass the textbook checks. With one early timer declared, the original resends a block, the receiver acknowledges both copies, and every later block then travels twice. This is the Sorcerer's Apprentice bug. With the same physics, the corrected sender has no such run; the check covers all 509 nodes of the model.

Bounded queues also make checking finite. Actors talking through unbounded queues can express undecidable problems (Brand and Zafiropulo, 1983). The textbook escapes this by restricting the protocol's shape. This version bounds the queues instead, as the hardware already does, so the number of states is finite.

5. Guarantees become questions with evidence

Finite actors and finite queues give a finite map of every state the system can reach. Any question about that map gets a yes or no, with a path as evidence: a witness for "this can happen", or a counterexample for "this must never happen".

The textbook's three theorems become three such questions. They are written in CTL, a standard notation for questions over branching futures: **A** means on every path, **E** on some path, **G** always, **F** eventually.

Textbook theorem	The same guarantee as a question
Communication safety	never: a message is waiting that its receiver has no handler for
Session fidelity	every run is a sentence of the grammar; also checked on recorded production logs
Progress	never: some actor waits, and nothing can ever move again, before the conversation ends

The textbook proves its theorems once, for every program that type-checks, under its channel assumptions. Here they are asked of one system with its declared physics, and any other question can be asked the same way:

- `AG (PayOrder → AF (Receipt ∨ PaymentDeclined))`: every payment eventually gets an answer.
- `EF SoldOut`: selling out is possible, so the test for the 409 is not dead code.
- `AG AF Idle`: the system always returns to idle; this needs a fairness assumption, stated as a requirement rather than hidden.

A counterexample is a concrete run, which can be drawn as a sequence diagram or replayed as a test.

6. An attacker is one more actor

The textbook assumes every party is honest and the channel is private. Security needs the opposite: an actor, here called Ivo, who *is* the network. Every message an honest actor sends goes to Ivo. Every message an honest actor receives is one Ivo could build.

What Ivo can build is decided by `knows(actor, value)`. It applies the standard Dolev–Yao rules: take apart pairs, open ciphertexts you hold the key for, and build pairs, encryptions and hashes from what you hold. Each state on the map records what Ivo knows, so secrecy is just another question.

The Needham–Schroeder protocol (1978) is three messages, and it is a perfectly well-formed session type. Ask:

```
never ( claimed commit(B, A) and knows(Ivo, nb) )
```

That is: Bob must never believe he is talking to Alice while Ivo holds Bob's secret nonce. LeanFM explores 228 states and returns a 10-state counterexample: Lowe's 1995 man-in-the-middle attack. With Lowe's fix, Bob's name added inside his reply, the same question is proved over all 220 reachable states.

The same function works for defenders. A filter in front of Bob can only block what `knows` lets it see. Without Bob's private key, an encrypted `bad` goes straight past it, and the counterexample shows the filter itself delivering it.

Translation table

Term	Plain version
OpenAPI document	a service: `sessions` of a `choice` between operations
OpenAPI operation	one protocol: a request, then a choice of responses
Response status code	one branch of that choice, tied to its request by `cause`
OpenAPI schema	a protobuf message, mirrored by a Lean structure
Global type	the protocol: a grammar whose words are messages
Role	an actor: a state machine with a bounded inbox
Projection	delete the messages an actor neither sends nor receives
Local type	one actor's part: its sends, receives and choices
Internal choice (⊕, select)	a choice this actor makes
External choice (&, offer, branch)	a choice another actor makes, learned from what arrives
Merge operator	rule 2: anyone whose future depends on a choice is told by a message
Realizability, non-local choice	rule 1: one actor starts every branch
Typing judgment, subject reduction	checking each actor's code against its part; checking logs against the grammar
Communication safety	never: a message waits that has no handler
Session fidelity	every run is a sentence of the grammar
Progress, deadlock freedom	never: stuck before the end
Asynchronous semantics	inboxes, with capacity and physics declared per channel
Duality (binary sessions)	for two actors, one's sends are the other's receives

What the simple version gives up

It trades generality for concreteness. Four things are lost:

- **Unbounded results.** A textbook theorem covers every number of sessions and any queue length. Here a “proved” means proved within the declared bounds: queue sizes, session counts, value domains.
- **Checking one piece at a time.** Type-checking looks at one process against its local type, so cost grows with the size of the code. Exploring the map looks at all actors together, and the number of states can grow very fast with the number of actors and the queue sizes.
- **Compile-time guarantees in a language.** Session-typed languages reject a bad program before it runs. Here conformance is checked against the protocol, a model of the code, and recorded logs. A program that never logs a bad run is not thereby proved correct.
- **General recursion.** Repetition is a tail-position loop, and data enters only through conditions on payloads. This keeps the map finite.

References

- OpenAPI Specification v3.1.0, OpenAPI Initiative, 2021.
- ProtoJSON Format, protobuf.dev: field names map to lowerCamelCase; 64-bit integers are encoded as strings.
- Honda, Yoshida and Carbone, Multiparty asynchronous session types, POPL 2008; journal version in JACM, 2016.
- Brand and Zafiropulo, On communicating finite-state machines, JACM, 1983.
- Bravetti, Carbone and Zavattaro, On the boundary between decidability and undecidability of asynchronous session subtyping, 2017.
- Lowe, An attack on the Needham–Schroeder public-key authentication protocol, Information Processing Letters 56, 1995.
- LeanFM: the notation, checks and verdicts used in this page; every example output is LeanFM′s own.